Quiz 6 - Introduction to Functions in Python

Ungraded in-class practice. Exam-style: code snippet + 5 true/false statements.

- Review: one question at a time
- Scope: full DataCamp course (Ch 1-3)
- Plan: 12 questions (Q11-Q12 are tricky review)

Question 1 (Chapter 1 - Defining a function, parameters, return, and docstrings)

Consider the following code:

```
def simple_return(start_price, end_price):
    """Return the simple price return."""
    return (end_price - start_price) / start_price

r = simple_return(100, 105)
print(r)
```

A. r is equal to 0.05.

B. `start_price` and `end_price` are parameters of the function.

C. `return` sends the calculated value back to the caller.

D. The docstring changes the numerical `result` produced by the function.

E. Calling `simple_return`(100) would work because `end_price` would automatically become 0.

Question 2 (Chapter 1 - Multiple parameters and keyword arguments)

Consider the following code:

```
def operating_profit(revenue, cogs, opex):
    return revenue - cogs - opex

profit = operating_profit(1000, cogs=400, opex=250)
print(profit)
```

A. profit is equal to 350.

B. `cogs`=400 and `opex`=250 are keyword arguments.

C. The keyword arguments could be written as `opex`=250, `cogs`=400 without changing the `result`.

D. `cogs` and `opex` are default parameters because their values are supplied in the function call.

E. The call `operating_profit`(`revenue`=1000, 400, 250) is valid Python syntax.

Question 3 (Chapter 1 - Multiple return values and tuples)

Consider the following code:

```
def trade_stats(entry_price, exit_price):
    pnl = exit_price - entry_price
    trade_return = pnl / entry_price
    return pnl, trade_return

result = trade_stats(50, 55)
print(result)
```

A. `result` is a tuple containing two values.

B. `result`[0] is equal to 5.

C. `result`[1] is equal to 0.10.

D. The assignment `pnl`, `ret` = `trade_stats`(50, 55) would be valid.

E. `result` is a list because the function `return`s more than one value.

Question 4 (Chapter 2 - Default arguments)

Consider the following code:

```
def after_tax_profit(ebit, tax_rate=0.25):
    return ebit * (1 - tax_rate)

base = after_tax_profit(200)
low_tax = after_tax_profit(200, 0.20)
print(base, low_tax)
```

A. base is equal to 150.0.

B. low_tax is equal to 160.0.

C. The default value 0.25 is used only when `tax_rate` is not supplied.

D. Calling `after_tax_profit`() with no arguments would work because both parameters have defaults.

E. `after_tax_profit`(200, `tax_rate`=0.30) would `return` 140.0.

Question 5 (Chapter 2 - Global and local scope)

Consider the following code:

```
risk_limit = 1000000

def set_limit():
    global risk_limit
    risk_limit = 750000

set_limit()
print(risk_limit)
```

A. After set_limit() runs, `risk_limit` is equal to 750000.

B. The `global` keyword makes the assignment refer to the `risk_limit` defined outside the function.

C. If the `global` line were removed, assigning `risk_limit` = 750000 inside the function would leave the outside `risk_limit` equal to 1000000.

D. A function must use `global` whenever it only reads a variable from `global` scope.

E. Declaring `risk_limit` as `global` makes the variable read-only inside the function.

Question 6 (Chapter 2 - Nested functions, nonlocal, and closures)

Consider the following code:

```
def make_budget_tracker(start_budget):
    budget = start_budget

    def spend(amount):
        nonlocal budget
        budget -= amount
        return budget

    return spend

tracker = make_budget_tracker(1000)
a = tracker(200)
b = tracker(150)
print(a, b)
```

A. a is equal to 800.

B. b is equal to 650.

C. `nonlocal budget` refers to `budget` in the enclosing `make_budget_tracker`() function.

D. `tracker` can remember the updated `budget` even after `make_budget_tracker`() has finished running.

E. Removing `nonlocal` while leaving `budget` -= amount unchanged would produce the same behavior.

Question 7 (Chapter 2 - Variable-length positional arguments (*args))

Consider the following code:

```
def total_pnl(*pnls):
    return sum(pnls)

day_pnl = total_pnl(120, -40, 70)
print(day_pnl)
```

A. Inside the function, `pnls` is a tuple.

B. day_`pnl` is equal to 150.

C. Calling `total_pnl`() with no arguments would `return` 0.

D. `sum`() is available here from Python built-in scope, so no import is required.

E. Calling `total_pnl`([120, -40, 70]) would `return` 150.

Question 8 (Chapter 2 - Variable-length keyword arguments (**kwargs))

Consider the following code:

```
def report_metrics(**metrics):
    return metrics

report = report_metrics(revenue=500, ebitda=120, debt=200)
print(report)
```

A. `report` is a dictionary.

B. `report`["ebitda"] is equal to 120.

C. **`metrics` allows the function to accept a variable number of keyword arguments.

D. Calling `report_metrics`(500, 120, 200) would create the same dictionary.

E. Every possible key passed to `report_metrics`() must be declared in advance in the function header.

Question 9 (Chapter 3 - Lambda functions and map())

Consider the following code:

```
prices = [100, 105, 110]
adjusted = list(map(lambda p: p - 1, prices))
print(adjusted)
```

A. `adjusted` is equal to [99, 104, 109].

B. `lambda` p: p - 1 defines a small anonymous function.

C. `map()` applies the `lambda` to each element of `prices`.

D. In Python 3, `map()` itself directly `return`s a list.

E. The original `prices` list is changed to [99, 104, 109].

Question 10 (Chapter 3 - Lambda functions and filter())

Consider the following code:

```
margins = [0.08, 0.15, 0.22, 0.05]
strong = list(filter(lambda m: m >= 0.15, margins))
print(strong)
```

A. `strong` is equal to [0.15, 0.22].

B. The `lambda` `return`s a Boolean value for each margin.

C. `filter()` keeps elements for which its function produces a truthy `result`.

D. The original `margins` list remains unchanged.

E. 0.08 appears in `strong` because it is a positive number.

Question 11 (Review - reduce() and cumulative calculations)

Consider the following code:

```
from functools import reduce

gross_factors = [1.02, 0.98, 1.05]
growth = reduce(lambda x, y: x * y, gross_factors)
print(growth)
```

A. reduce must be imported from `functools` in this example.

B. `growth` is approximately equal to 1.04958.

C. `reduce()` repeatedly combines values until one accumulated `result` remains.

D. Replacing reduce(...) with `sum`(`gross_factors`) would produce the same `result`.

E. reduce(`lambda` x, y: x * y, [1.02]) `return`s 1.02.

Question 12 (Review - try-except and raising errors)

Consider the following code:

```
def debt_ratio(debt, assets):
    if assets <= 0:
        raise ValueError("assets must be positive")
    return debt / assets

try:
    ratio = debt_ratio(300, 0)
except ValueError:
    ratio = None

print(ratio)
```

A. `debt_ratio`(300, 0) raises a `ValueError`.

B. The except block catches that `ValueError` and sets ratio to `None`.

C. After raise `ValueError`(...) executes, the function continues to `return` debt / assets.

D. `debt_ratio`(300, 600) would `return` 0.5.

E. Without the `try-except` block, `debt_ratio`(300, 0) would quietly `return` `None`.